

Diário de Bordo — Sistema de Reservas de Equipamentos Tecnológicos

Estrutura de Dados I — Trabalho Prático I — UFPR (TADS)

Autor: João Victor Timoteo (trabalho individual)

Professor(a): HELCIO PADILHA

Repositório: <https://github.com/joaovt-ufpr/reservas-equipamentos>

1. Quem fez o quê

O trabalho foi feito por mim, sozinho. Cuidei de todas as partes: o programa em C (menu, listas e funcionalidades), os testes, o README, este diário e o repositório no GitHub.

2. Ferramentas que usei

- Linux, editor VS Code, compilador gcc e Git/GitHub.
- Material e aulas da disciplina de Estrutura de Dados I.
- Usei o Claude (assistente de IA) como apoio: para montar a primeira versão do código, tirar dúvidas e organizar a documentação. Depois estudei o código para conseguir explicá-lo e testá-lo por conta própria.

3. Como entendi o problema

O programa precisa guardar equipamentos (notebook, tablet, câmera, projetor) e as reservas de cada um, sem deixar duas pessoas reservarem o mesmo equipamento no mesmo horário. O histórico precisa ser separado por tipo de equipamento.

Pensei assim: cada tipo tem uma “ficha” (o cabeçalho) com o nome do tipo e a contagem de equipamentos e reservas. Presa a essa ficha vêm duas listas: a dos equipamentos daquele tipo e a das reservas daquele tipo. Escolhi lista encadeada porque não sei quantos itens vão existir, e ela vai crescendo conforme eu cadastro.

4. Relato semanal

Semana 1 —

O que fiz

- Li o enunciado e anotei as seis funcionalidades que o programa precisa ter.
- Instalei e configurei o ambiente: compilador, VS Code e Git.
- Desenhei no papel como as listas se ligam (tipos, equipamentos e reservas).
- Estudei como funciona uma lista encadeada (nó, ponteiro prox, início e fim da lista) e como o cabeçalho organiza uma lista por tipo.
- Li cada arquivo do código (lista.c, equipamentos.c e main.c) para entender o que cada função faz.
- Compilei e rodei o programa pela primeira vez, testando o menu.

Dificuldades e como resolvi

- O terminal do VS Code não respondia quando eu digitava as opções do menu. Resolvi abrindo o terminal normal do Linux e usando um arquivo de teste (teste_entrada.txt), que digita as opções por mim.
- A primeira versão do programa ficou mais complicada do que eu conseguia explicar. Decidi simplificar e ficar com o que o enunciado pede.
- No começo foi difícil entender como o cabeçalho se liga às duas listas. Ajudou desenhar no papel uma caixa por tipo, com duas “correntes” de itens saindo dela.

Uma parte do código explicada com minhas palavras

Função `inserir_equipamento`, que coloca um equipamento novo na lista do tipo:

```
if (cab->equipamentos == NULL) {
    cab->equipamentos = novo;
} else {
    Equipamento *e = cab->equipamentos;
    while (e->prox != NULL)
        e = e->prox;
    e->prox = novo;
}
cab->qtdEquipamentos++;
```

Primeiro eu vejo se a lista desse tipo está vazia. Se estiver, o equipamento novo passa a ser o primeiro. Se já tiver itens, eu começo no primeiro e vou “andando” de um em um até chegar no último (o que não aponta para ninguém). Aí ligo o novo equipamento no fim. No final somo 1 no contador

que fica no cabeçalho, assim sei quantos equipamentos o tipo tem sem precisar contar de novo.

Semana 2 —

O que fiz

- Rodei os testes de cada funcionalidade: cadastrar, reservar, conflito de horário, listar por tipo, consultar por data, cancelar e histórico.
- Conferi que o programa libera a memória (usei o valgrind).
- Escrevi o README, completei este diário e criei o repositório no GitHub.
- Treinei a explicação das funções principais para a apresentação oral.

Dificuldades e como resolvi

- Entender por que cancelar uma reserva não apaga o item da lista: ele fica marcado como cancelado para o histórico continuar completo.
- Passar o projeto para o GitHub e preparar a apresentação no Windows, já que desenvolvi no Linux.

Uma parte do código explicada com minhas palavras

Função buscar_reserva_ativa, que descobre se um equipamento já está reservado:

```
Reserva *r = cab->reservas;
while (r != NULL) {
    if (r->ativa && r->codigoEquip == codigo &&
        strcmp(r->data, data) == 0 && strcmp(r->hora, hora) == 0)
        return r;
    r = r->prox;
}
return NULL;
```

Ela olha uma por uma as reservas do tipo do equipamento. Para cada reserva, pergunta: ela ainda está ativa? É do mesmo equipamento? É no mesmo dia e na mesma hora? Se as respostas forem todas “sim”, achei um conflito e devolvo essa reserva. Se terminar a lista sem achar nada, devolvo NULL, que quer dizer “horário livre”. Como só olho reservas ativas, um horário cancelado pode ser reservado de novo. Uso a mesma função para achar a reserva que o usuário quer cancelar.

5. Resultado

O programa faz as seis funcionalidades pedidas e foi testado com dados corretos e errados. Ele não salva os dados em arquivo (tudo some quando fecha) e só confere se a data está no formato DD/MM/AAAA.

6. O que aprendi

Aprendi como funciona uma lista encadeada: cada item aponta para o próximo, e para chegar ao fim é preciso andar item por item. Entendi para que serve o cabeçalho: ele guarda informações do grupo (nome do tipo e contadores) e o começo das listas, o que deixa o histórico separado por tipo. Vi que aqui a lista é melhor que um vetor porque não sei quantos equipamentos e reservas vão existir. Também aprendi que é melhor marcar a reserva como cancelada do que apagá-la, para não perder o histórico, e que é preciso liberar a memória no final do programa. Por fim, aprendi o básico de Git e GitHub para entregar o trabalho.